



Universidad Autónoma de Baja California

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA



Programación orientada a objetos.

Laboratorio No. 4: Clases.

Alumno:

Joshua Osorio O. – 1293271

Docente:

Gustavo O. Zamarron De La Garza

Septiembre/2026

Tabla de contenido

I.	Objetivo	3
II.	Descripción de la practica.....	3
	Lista de materiales.....	3
III.	Desarrollo de la práctica	4
	Metodología y Arquitectura	4
	Descripción del problema y elementos clave	4
	Datos de entrada y datos de salida	4
	Diagrama de clases.....	5
	Fragmentos de código y visualización de elementos clave	6
	Clase CuentaBancaria	6
	Clase Banco	6
	Clase Cajero	6
	Clase Main — menú principal y submenús	7
	Reflexión y análisis crítico	8
	Dificultades técnicas.....	8
	Fracasos, fallas, bugs y errores	8
	Cómo se resolvió.....	8
	Conclusión	8
IV.	Bibliografía.....	9

I. Objetivo

El alumno aplicará los conceptos de programación orientada a objetos (clases, atributos privados, constructores, encapsulamiento mediante getters/setters y colaboración entre objetos) para modelar y simular un cajero electrónico: una clase CuentaBancaria que administra el saldo y el historial de una cuenta, una clase Banco que administra el conjunto de cuentas, y una clase Cajero que implementa las operaciones disponibles para el usuario (depositar, retirar, eliminar cuenta e imprimir recibo) sobre la cuenta con sesión iniciada.

II. Descripción de la practica

Haz el código del siguiente problema.

1.- Realiza un programa donde simule un cajero electrónico y sus operaciones que puedes realizar en una cuenta bancaria.

A) La clase CuentaBancaria contiene

Numero de cuenta, titular, saldo.

operación de depositar y de retirar

operación de saldar cuenta

B) Un cajero contiene:

Operación de Ingresar,

Operación de depósito, retiro, eliminar cuenta

Operación de imprimir recibo

Operación de salir

El cajero cada vez que haces un depósito o retiro o eliminar cuenta , debe de imprimir un recibo con la operación que se hizo y el historial del saldo de la cuenta, así como información sensible de la cuenta.

Lista de materiales

- Computadora personal (PC).
- JDK (Java Development Kit) y compilador javac.
- Editor de texto / IDE (Visual Studio Code) y terminal con acceso a git.

III. Desarrollo de la práctica

Metodología y Arquitectura

Descripción del problema y elementos clave

En este laboratorio, gran parte del análisis del problema también venía resuelto de antemano: la especificación indicaba explícitamente qué clases debían crearse (CuentaBancaria y Cajero) y qué atributos y operaciones debía tener cada una. Esto dejó como tarea principal el diseño de la implementación: cómo repartir la lógica entre CuentaBancaria, Banco (la colección de cuentas) y Cajero (las operaciones de cara al usuario), y cómo estructurar un menú que permitiera iniciar sesión y operar sobre una cuenta desde un único punto de entrada (Main).

Los elementos clave del diseño son los siguientes:

- CuentaBancaria, Banco y Cajero se implementaron como tres clases independientes con responsabilidades separadas: CuentaBancaria concentra los datos y la lógica de una cuenta (saldo, historial, depositar/retirar/saldar); Banco administra el arreglo de cuentas (crear, buscar, iniciar sesión, eliminar); y Cajero implementa las operaciones que ve el usuario (depositar, retirar, eliminar cuenta, imprimir recibo), delegando en Banco y en la CuentaBancaria en turno.
- Main es la clase que contiene el método main y no está pensada para instanciarse: declara como atributos estáticos un Scanner compartido, el Banco y el Cajero del programa, de modo que los métodos estáticos del menú puedan usarlos sin recibirlos como parámetro.
- El menú se organizó en una pantalla de inicio (iniciar sesión / crear cuenta / cargar cuentas de prueba) y, ya con sesión iniciada, un menú del cajero con las operaciones sobre la cuenta en turno, en vez de un único menú plano con todo mezclado.

Datos de entrada y datos de salida

Datos de entrada

- Pantalla de inicio: número entero para elegir la opción (1 iniciar sesión, 2 crear cuenta, 3 cargar cuentas de prueba, 0 salir).
- Iniciar sesión: número entero de cuenta.
- Crear cuenta: nombre del titular (texto) y saldo inicial (double).
- Menú del cajero (con sesión iniciada): número entero para elegir la operación (1 depositar, 2 retirar, 3 eliminar cuenta, 4 imprimir recibo, 0 salir), y montos double para depósito/retiro.

Datos de salida

- Mensajes de confirmación y error (por ejemplo, "Cuenta creada con éxito...", "Error: saldo insuficiente para realizar el retiro.").

UABC_FCQI_PROGRAMACIÓNORIENTAOBJETOS.

- El recibo del cajero: operación realizada, monto, titular, número de cuenta enmascarado, saldo actual e historial completo de movimientos.
- Mensajes de navegación del menú (bienvenida al iniciar sesión, sesión cerrada, opción no válida, saliendo del programa).

Diagrama de clases

El siguiente diagrama muestra las cuatro clases del programa: Main, que usa (dependencia, flecha punteada) tanto a Banco como a Cajero; Banco, que administra (agregación) el arreglo de objetos CuentaBancaria; y Cajero, que también usa CuentaBancaria para operar sobre la cuenta en turno. Main no aparece conectado por herencia ni composición fuerte porque solo usa a Banco y Cajero como colaboradores.

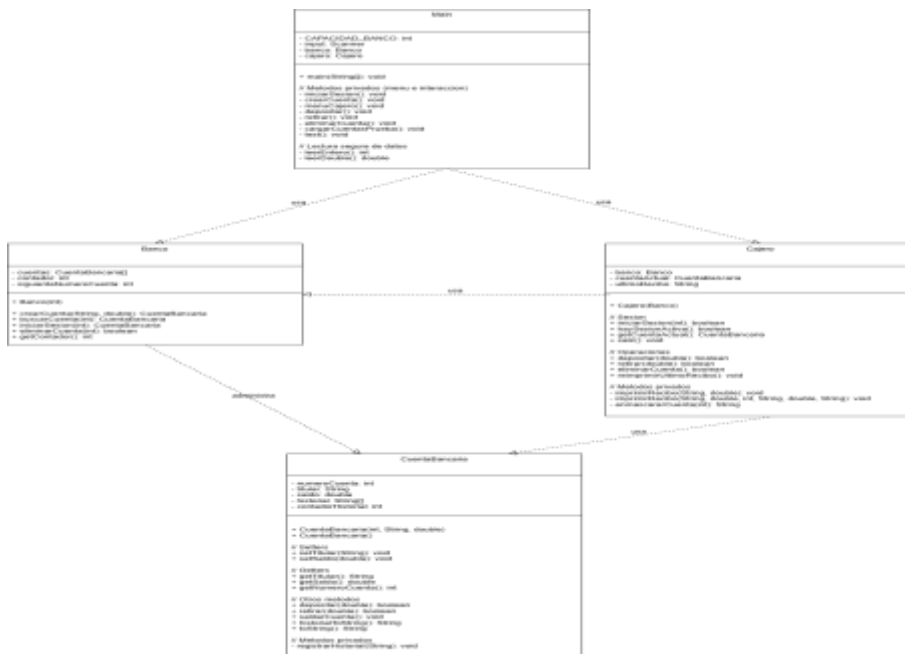


Ilustración 1: Main, Banco, Cajero y CuentaBancaria.

Fragmentos de código y visualización de elementos clave

Clase CuentaBancaria

```
public boolean depositar(double monto) {
    if (monto <= 0) return false;
    saldo += monto;
    registrarHistorial(...);
    return true;
}
```

Programé depositar() y retirar() como métodos que primero validan el monto (debe ser mayor a 0, y en retirar() además no puede exceder el saldo) antes de modificar el saldo, y en ambos casos registro el movimiento en el historial con registrarHistorial(). saldo lo mantengo como double para poder manejar centavos, igual que en el resto del curso.

```
Cuenta #9999 | Titular: Cuenta de prueba | Saldo: $1000.00
Depositar 500 (valido):
Cuenta #9999 | Titular: Cuenta de prueba | Saldo: $1500.00
Retirar 300 (valido):
Cuenta #9999 | Titular: Cuenta de prueba | Saldo: $1200.00
Retirar 999999 (mayor al saldo, debe rechazar):
Error: saldo insuficiente para realizar el retiro.
```

Clase Banco

```
public CuentaBancaria crearCuenta(String titular, double saldoInicial) {
    if (contador >= cuentas.length) return null;
    CuentaBancaria nueva =
        new CuentaBancaria(siguieteNumeroCuenta, titular, saldoInicial);

    cuentas[contador++] = nueva;
    siguieteNumeroCuenta++;
    return nueva;
}
```

Para Banco usé un arreglo de tamaño fijo (CuentaBancaria[]) en vez de una lista dinámica, siguiendo la misma convención de arreglos de tamaño fijo del resto del curso; contador lleva cuántas cuentas hay realmente ocupadas del arreglo, y siguieteNumeroCuenta es un folio autoincremental que empieza en 1000 para asignar números de cuenta únicos sin que el usuario los capture.

```
Crear cuenta valida (Juan Perez, saldo 2000):
Cuenta #1000 | Titular: Juan Perez | Saldo: $2000.00
Eliminar cuenta existente (1001):
Eliminada: true | Cuentas restantes: 1
Eliminar cuenta inexistente (55555):
Eliminada: false
```

Clase Cajero

```
public boolean retirar(double monto) {
    if (!haySesionActiva()) return false;
    boolean exito = cuentaActual.retirar(monto);
}
```

```

    if (exito) imprimirRecibo("Retiro", monto);
    return exito;
}

```

Cajero no duplica la lógica de negocio: delega el depósito y el retiro directamente en la CuentaBancaria en turno (cuentaActual), y solo se encarga de verificar que haya una sesión activa y de imprimir el recibo cuando la operación tuvo éxito. La sobrecarga de imprimirRecibo() (una versión corta para depósito/retiro y otra con todos los datos explícitos para la eliminación de cuenta, cuando la cuenta ya fue borrada de Banco) evitó repetir el formato del recibo dos veces.

```

=====
                        RECIBO DEL CAJERO
=====
Operacion:      Deposito
Monto:          $700.00
-----
Informacion de cuenta -----
Titular:        Juan Perez
No. de cuenta:  **00
Saldo actual:   $2700.00
-----
Historial -----
1. Apertura de cuenta. Saldo inicial: $2000.00
2. Deposito: +$700.00 -> Saldo: $2700.00
=====

```

Clase Main — menú principal y submenús

```

public static void main(String[] args) {
    int opcion;
    do {
        // ...imprime el menu de inicio...
        opcion = leerEntero();
        switch (opcion) {
            case 1: iniciarSesion();      break;
            case 2: crearCuenta();        break;
            case 3: cargarCuentasPrueba(); break;
            case 0: System.out.println("Saliendo del programa..."); break;
            default: System.out.println("Opcion no valida, intente de nuevo.");
        }
    } while (opcion != 0);
    input.close();
}

```

Para Main, el diseño del menú también fue lo más tedioso de decidir: opté por una pantalla de inicio (sin sesión) y un menú del cajero aparte (con sesión), en vez de un único menú con todas las operaciones juntas, para que las opciones que requieren una cuenta iniciada (depositar, retirar, eliminar, imprimir recibo) no aparezcan disponibles antes de iniciar sesión. banco y cajero se declararon como atributos estáticos de Main para que todos los métodos del menú compartieran la misma instancia sin pasarla como parámetro. Además, agregué una opción de prueba oculta (test(), no impresa en el menú pero accesible tecleando 4) que ejercita automáticamente todas las operaciones de CuentaBancaria, Banco y Cajero — válidas e inválidas — con datos fijos, sin necesidad de capturar todo manualmente cada vez.

```

=====
                        BANCO - INICIO
=====
1 -      Iniciar sesion
2 -      Crear cuenta nueva
3 -      Cargar cuentas de prueba
0 -      Salir del programa

```

Reflexión y análisis crítico

Dificultades técnicas

La dificultad técnica más importante fue diseñar la interacción entre las cuatro clases sin duplicar lógica: decidir qué le corresponde a CuentaBancaria (sus propios datos y validaciones), qué a Banco (la colección y el ciclo de vida de las cuentas) y qué a Cajero (solo las operaciones de cara al usuario, delegando el resto), en vez de meter toda la lógica en Main. Otra dificultad fue el manejo del historial de movimientos con un arreglo de tamaño fijo (no una lista dinámica), que obligó a decidir qué hacer cuando el arreglo se llena (se recorre para conservar solo los movimientos más recientes).

Fracasos, fallas, bugs y errores

El primer error fue de diseño, no de sintaxis: en un intento inicial, Cajero manipulaba directamente el arreglo de cuentas de Banco, lo que rompía el encapsulamiento y hacía que cualquier cambio en Banco obligara a tocar Cajero también. También tuve, igual que en prácticas anteriores, el problema clásico de mezclar `input.nextInt()/nextDouble()` con `input.nextLine()` en el mismo Scanner, ya que `nextInt()/nextDouble()` no consumen el salto de línea, lo que hacía que la siguiente captura de texto (por ejemplo, el nombre del titular) se saltara.

Cómo se resolvió

El problema de encapsulamiento se resolvió dándole a Cajero una referencia al Banco (no al arreglo interno) y dejando que sea Banco quien exponga los métodos necesarios (`buscarCuenta`, `iniciarSesion`, `eliminarCuenta`), de modo que Cajero nunca toca el arreglo directamente. El problema del Scanner se resolvió centralizando la lectura en dos métodos (`leerEntero()`, `leerDouble()`) que siempre consumen la línea sobrante con `input.nextLine()` después de leer el número, y que además atrapan `InputMismatchException` para pedir de nuevo el valor si el usuario captura texto donde se esperaba un número.

Conclusión

Esta práctica ayudó a practicar el reparto de responsabilidades entre varias clases colaborando entre sí (CuentaBancaria, Banco, Cajero y Main), en vez de una sola clase con todo el comportamiento, cuidando además qué información expone cada una (por ejemplo, el número de cuenta enmascarado en el recibo, o el borrado de datos sensibles al saldar una cuenta) y cómo se refleja eso en un menú de consola organizado en pantalla de inicio y menú del cajero.

IV. Bibliografía

- [1] Oracle Corporation. (2024). Java SE Documentation. Recuperado de <https://docs.oracle.com/javase/>
- [2] Deitel, P. J., & Deitel, H. M. (2016). Java: How to Program (10th ed.). Pearson Education.